Домашнее задание 2. Web.API.FastAPI.

Формат сдачи:

Все домашние задания по курсу сдаются как страничка в Github-репозитории, который структурирован так, как указано в конце задания ниже. Пожалуйста, обратите внимание на взаимное расположение файлов в репозитории и на их название. Я буду запускать автоматический базовый чекер, который проверяет наличие файлов в репозитории для каждой домашки. Для каждого задания старайтесь хранить листинг команд, которыми вы пользовались – они понадобятся для сдачи. Часто ответы на задание нужно будет писать в README.md в какой-то папке репозитория (воспринимайте это как что-то похожее на домашки по практической биоинформатике).

Задание 5 будет проверяться в два этапа: до моего PR в ваш репозиторий и после. См. задание для уточнения процесса сдачи.

Уточнение:

Если у вас не Linux, вы можете воспользоваться либо WSL2 (устанавливается автоматически в Windows > 10), либо установить cygwin: **cygwin.org**, позволяющий работать в командной строке. По всем вопросам пишите в чат курса или в личные сообщения преподавателям. При работе с git не забывайте как можно чаще коммитить изменения. В markdown можно вставлять картинки и таблицы – не стесняйтесь это делать, если чувствуете, что это хорошо иллюстрирует ситуацию.

Структура репозитория после второго ДЗ:

```
.
├── .github
│   └── workflows
│       ├── PR-tg-notify.yaml
│       └── ...
├── .gitignore
├── hw1
│   └── ...
├── hw2
│   ├── wikipedia_articles.py
│   ├── draw_wiki.py
│   ├── selenium_amazon_crawl.py
│   ├── draw_amazon.py
│   ├── get_domains.py
│   ├── get_variable_splicing.py
│   ├── checkfile.yaml
│   ├── message.txt
│   ├── RPI_server.py
│   ├── ...
│   └── README.md
├── hw3
│   └── ...
├── hw4
│   └── ...
├── hw5
│   └── ...
└── README.md
```

Задание 1. Веб-кроулер на Wikipedia.

Реализуйте веб-кроулер **`hw2/wikipedia_articles.py`**, который, почти как в лекции, принимает точку входа-URL статьи в Wikipedia (аргумент **`--url`**), глубину (аргумент **`--depth`**). Кроулер начинает работу в исходной статье на Wikipedia, извлекает из её тела все внутренние ссылки на другие статьи Wikipedia, затем переходит по этим ссылкам до заданной глубины, при этом избегая повторного посещения одних и тех же страниц. В конце кроулер должен озвращает результат в виде **графа связей** (в формате JSON) в файл **`start_article.json`** (**`start_article`** – название, title исходной страницы, с которой начинался кроулинг). С помощью библиотеки networkX нарисуйте граф связей, который получили из JSON для своей любимой статьи на Wikipedia и **`depth=5`**. Сохраните скрипт для отрисовки в **`hw2/draw_wiki.py`**, в **`hw2/README.md`** вставьте картинку и название начальной статьи.

Задание 2. Веб-драйвер на Amazon.

Реализуйте кроулер **`hw2/selenium_amazon_crawl.py`** с помощью веб-драйвера Selenium для обработки страницы поиска интернет-магазина Amazon с определённым запросом (аргумент **`--query`**). Вы можете пользоваться кодом из лекции, но вам придётся добавить обработку пагинации, парсинг результатов и поменять драйвер, если у вас стоит не браузер Firefox. Веб-драйвер должен прогрузить первые 5 страниц поиска для Amazon по поисковому запросу, собрать с них все наименования и теги цены для найденных товаров, и записать их в JSON в файл **`query.json`** (**`query`** – переданный в код параметр запроса). С помощью графических библиотек в питоне нарисуйте гистограмму распределения цен на товары (обязательно укажите валюту), которые нашлись. Примеры запросов: **`labubu`**, **`RTX4090`**, **`windows`**, **`stanley`**, можно и другие. Сохраните скрипт для отрисовки в **`hw2/draw_amazon.py`**, в **`hw2/README.md`** вставьте картинку, текст запроса.

Задание 3. REST API в ENCODE.

Изучите API, предоставляемый базой данных **ENCODE**. С помощью скрипта на python (**`hw2/get_domains.py`**) обратитесь к REST API ENCODE. Вам нужно получить список белков, для которых в этой базе данных проведён эксперимент типа **`TF ChIP-seq`** на клеточной линии (именно **`cell line`**), для которой в этой БД существует наибольшее количество экспериментов типа **`DNAse-seq`** (при этом, очевидно, данные **`TF ChIP-seq`** для этой клеточной линии <u>тоже должны существовать</u>). Для полученных белков в файл **`domains.json`** выведите список названий доменов в InterPro, которые содержатся в этом белке. Указание: для этого вам понадобится сматчить **gene symbols** белков, которые вы получили из ENCODE, с идентификаторами InterPro (UniProt).

Итоговый пайплайн выглядит так:

1. Запрос к ENCODE для получения клеточных линий, у которых есть TF ChIP-seq и DNAse-seq;
2. Запрос к UniProtKB или API InterPro для получения и матчинга символов генов общепринятым в InterPro символам белков.;
3. Запрос к Pfam (InterPro API) для получения названий доменов каждого белка.

В **`hw2/README.md`** запишите то, какими полями и в каком API вы пользовались. Для API InterPro перечислите использованные вами эндпоинты.

Задание 4. Обращение к ensembl GraphQL.

Изучите документацию GraphQL API для ensembl. Запросите все белок-кодирующие гены на какой-нибудь человеческой хромосоме (на ваш выбор, чем меньше хромосома, тем быстрее отработает запрос). Отберите только те белки, у которых есть несколько изоформ альтернативного сплайсинга, причём альтернативный сплайсинг затрагивает количество доменов в получающемся белке (т.е., количество белковых доменов Pfam у изоформ различается). ID сборки человека: **`“2b5fb047-5992-4dfb-b2fa-1fb4e18d1abb”`**.

1. **Формулирование запроса ко всем генам с хромосомы.**
   Для этого можно воспользоваться схемой **`overlap_region by_slice`**. Внимание: graphQL ensEMBL может бросать ошибки, если вы передадите в него регион больше 4 Мбп, поэтому вам придётся организовать некую пагинацию, сделав несколько последовательных запросов по длине хромосомы.
   В результате запроса вам понадобится несколько рекурсивных вызовов к схемам. На верхнем уровне получите схему **`genes`** (возьмите в ней нужные вам поля – скорее всего, это **`symbol`**, **`stable_id`** и **`metadata.biotype.label`** для фильтрации белок-кодирующих генов), в ней получите **`genes.transcripts`** (в ней тоже есть нужные вам поля, такие как **`stable_id`**). Далее, внутри этой схемы вам понадобится **`product_generating_contexts.product`** – схема, в которой хранятся метаданные того, как из гена получается продукт. В ней есть **`family_matches.sequence_family`** – как раз информация о Pfam. Попробуйте сами собрать запрос в graphQL.
2. **Обработка JSON с помощью requests и питона.**
   Внутри результирующих прилетающих из API JSON вытащите оставьте только те белки, у которых для изоформ различаются количества уникальных доменов Pfam в них. В файл **`variable_spliceforms.json`** выведите JSON с названием белка (symbol), его ENSG стабильным идентификатором и количеством найденных изоформ с разным набором доменов. Напишите код решения **`hw2/get_variable_splicing.py`**, принимающий номер хромосомы как аргумент **`--chrom`**. Длины хромосом можно захаркодить внутри кода словарём.

В **`hw2/README.md`** запишите то, какие поля в GraphQL вы использовали и что фильтровали по итогу в результатах запроса к ensembl. Кроме того, поместите в **`README.md`** целиком GraphQL-запрос.

Задание 5. Telegram-уведомления для Github Actions.

Разберитесь, как хранить секреты в Github. Это важно, потому что мы не хотим, чтобы ваши репозитории пополняли ряды тех многочисленных Github-проектов, в которых токены к каким-то приложениям закоммичены в репозиторий. Положить секреты для репозитория можно в **Settings -> Secrets and variables -> Actions**. Если вы положите туда API-токен для телеграм-бота, он никому не будет виден, кроме коллаборaторов (обещаю, что мне ваш токен не нужен). Проверьте, что видимость репозитория – публичная.

Теперь к заданию. Напишите систему логирования вашего репозитория Github для случая, когда кто-либо не под вашим аккаунтом создаёт pull-request в ваш репозиторий. Для этого вам нужно будет написать ещё один Actions-воркфлоу. Для этого вам надо будет разобраться с тем, какие бывают в Github переменные для действий:

- **События, которые триггерят workflow**;

- **Структура событий веб-хуков** (для actions они те же);
- **Примеры событий для pull-request**.

Для отправки API-запросов вы можете использовать библиотеку requests (тогда вам надо разобраться с тем, как правильно отправлять payload на API телеграма по адресу **api.telegram.org**), либо использовать те же модули, что были рассмотрены на лекции. Обращаю ваше внимание, что в любом случае и **`requests`** и **`python-telegram-bot`** надо отдельным шагом устанавливать в воркфлоу, иначе он не отработает.

Создайте в репозитории файлы **`hw2/checkfile.yaml`** и **`hw2/message.txt`** (пустой). В первом файле должна быть только пара ключ-значение **`{checked: False}`**, во второй файл я при проверке допишу текст. После проверки (когда придёт мой PR в hw2) в **`hw2/README.md`** вставьте скриншот чата с вашим ботом, где видно уведомление от него. Формат уведомления смотрите ниже.

Ваш воркфлоу должен по приходу нового PR в репозиторий отправлять вам в личные сообщения сообщение свободного формата, в котором должны содержаться поля, как в примере ниже (другие поля тоже можно включать, но те, что ниже, обязательны).

```
Новый PR от @username!
    Название: "Pull Request to test workflow for <user>"
    Дата и время создания: 2026-04-13 18:45:40
    Ссылка: https://github.com/your_login/repo/pull/123
    Изменённые файлы:
      - hw2/checkfile.yaml
      - hw2/message.txt
    Текст в message.txt:
      “”“
      Тут будет что-то написано мной.
      “”“
```

За форматирование сообщения (жирный шрифт, кликабельные ссылки и т.д.) плюс в карму.

Задание 6. Интерфейс FastAPI.

Напишите серверную модель API с помощью фреймворка FastAPI для предсказания взаимодействия РНК и белка. Для начала, возьмите 20 белков из ENCODE (можно из предыдущего задания), и придумайте (сгенерируйте случайно с сохранением сида) каждому из них 40 РНКовых мотивов длиной от 3 до 9 нуклеотидов в алфавите ATGC. Каждому мотиву сопоставьте случайный скор от -1 до 1. Должен получиться JSON или CSV-файл примерно такого формата:

JSON	CSV

`{`
`  “EZH2”: {“AGTG”: -0.3,`
`  …,`
`  “GGAGC”: 0.75},`
`  …`
`}`
	`EZH2,ATGT,-0.3`
	`EZH2,…,…`
	`…`
	`EZH2,GGAGC,0.75`
	`…`

Этим JSON должен будет пользоваться ваш API-шлюз, который реализован с помощью FastAPI.

У API должно быть два эндпоинта:

1. **/proteins**

   При запросе GET этого эндпоинта возвращается json белков, для которых поддерживается инференс модели. Возвращайте только JSON-список имён белков. Для обработки того, что отправляете рекомендую использовать pydantic модели данных.

2. **/predict**

   Этот эндпоинт должен поддерживать метод POST. Эндпоинт должен ожидать, что на него придёт запрос с JSON-payload в виде списка или одной пары РНК-Белок в формате **`[{"protein": protein, "gene_name" : name, "sequence": ACTACGT…ACTAU}]`** – обратите внимание, что если отправлен T или U, ваш сервер не должен упасть, но вернуть обработанное сообщение о том, что все U конвертированы в T как дополнительное поле возвращаемого JSON. Эндпоинт возвращает JSON, где каждой паре gene_name - protein сопоставлен скор (score) взаимодействия как сумма скоров белка по мотивам РНК, если они есть в ней. Если в РНК нет мотивов, известных по белку, оправляйте None. Если мнемоника белка отсутствует в ваших исходных данных, отправляйте None. Пример запроса и базы:

JSON	POST-запрос (payload)

`{`
`  "EZH2": {"AGTG": -0.3,`
`  "GGAGC": 0.75},`
`}`
	`{[`
	`  {"protein": "EZH2",`
	`  "gene_name": "HOTAIR",`
	`  "sequence": "AGTGGAGCAGTGAGTG"},`
	`  {"protein": "SUZ12",`
	`  "gene_name": "HOTAIR",`
	`  "sequence": "AGTGGAGCAGTGAGTG"},`
	`]}`

Пример возвращаемого JSON для этого запроса и этой базы данных:

RESPONSE (JSON)

`{[`
`  {"protein": "EZH2",`
`  "gene_name": "HOTAIR",`
`  "score": "-0.15",`
`  "notes": null},`
`  {"protein": "SUZ12",`
`  "gene_name": "HOTAIR",`
`  "score": null,`
`  "notes": "Error: No such protein in DB"},`
`]}`

Положите код приложения в **`hw2/RPI_server.py`**. Если вы использовали другие файлы и модули для работы API, которые подключаются отдельно, укажите об этом в **`README.md`**. Кроме того, в **`README.md`** опишите то, как работает сервер (какие вы проставили fallback'и, обработки ошибок и другое). Опишите, как вы тестировали ваш API. Добавьте в **`hw2/README.md`** скриншот автоматически сгенерированной OpenAPI документации.